

Classification: Simulation Laboratory Planning Document – Internal

**Version 1.0 · Prepared 16 August 2026 · Prepared for Omniversal Architect Labs / Law
Laboratory AI Collaboration Team**

**Preparer: Independent senior research analyst and red-team methodologist synthesis
(Perplexity deep research pass)**

Table of Contents

1. Executive Summary
2. Development Track Record
3. Mission Statement, Purpose, Objectives, and Core Research Questions
4. Laboratory Architecture Recommendation
5. Red-Team Analysis
6. Unified Glossary (Proposed)
7. Proposed Failure & Disposition Taxonomy
8. Pilot Experiment Specification (L-001)
9. Phased Development Roadmap
10. Governance Recommendations
11. Recommendations & Decision Items for the Team
12. Appendix: Contribution Cross-Reference

1. Executive Summary

Current maturity of the concept. The Law Laboratory has progressed from a raw, "bare bones" idea into a concept that has survived two independent rounds of multi-AI formalization without collapsing into incoherence, contradiction, or metaphysical overreach. Six independent systems in the First Response Pass-Through arrived at nearly identical experimental logic (define a law, place it in an environment, escalate it, attack it, classify the result). Five independent reviewers in the Decision and Red Team Pass then converged on the same next-stage diagnosis: the methodology is settled, but the vocabulary, the definition of "law," the treatment of dependencies, and the governance model are not. The project is therefore ready to move from open-ended conceptual rewriting into a locked charter and a single executable pilot — not into full-scale transfinite or ascension-family experimentation.^[1]
^[2]

Strongest areas of multi-AI convergence. Three points of convergence are unanimous across all eleven contributor passes (six first-round, five second-round): the laboratory makes no claims about base reality and exists solely to map internal, formally specified rule behavior; "no dependencies" cannot be a literal, absolute property of any executable simulation and must instead be reframed as a graded, testable condition; and negative results (a law breaking,

requiring a dependency, or hitting a ceiling) are primary scientific outputs, not project failures. A fourth strong convergence, identified specifically in the second pass, is that the research cycle itself — specify, instantiate, escalate, challenge, classify, archive — is essentially the same across Grok, Perplexity, Copilot, Claude, Gemini, and ChatGPT, differing only in the number of named layers.^{[2][1]}

Highest-priority unresolved decisions. Five decisions must be locked before any code is written: what counts as a "law" (permission, obligation, constraint, transformation, escalation, inheritance rule, meta-law, or operator); whether "no dependencies / no triggers" is a universal constraint, a test condition, a candidate law itself, or a null hypothesis; what must persist for a transformed entity to count as a continuation rather than a successor (the inheritance/identity problem); how simulator/implementation failure is distinguished from genuine law failure; and who adjudicates when independent red-team probes on the same law disagree.^{[1][2]}

Recommended immediate next step. Do not commission another open-ended conceptual rewrite. The team should instead lock the unified glossary and authority hierarchy proposed in this report, ratify (or amend) the L-001 pilot specification in Section 8, and execute a single minimal, fully instrumented run before any recursive, cascading, or transfinite work begins.
[[^]2]

2. Development Track Record

2.1 Origin (informal concept)

The project originated from a personal synthesis of prior, fragmentary development work: a "Law of Infinite Ascension," multiple evolutionary principle variants (traditional, recursive, non-linear, cascading, unbounded), and ascension variants pushed toward transfinite, transcardinal, and uncountable extremes, alongside a standing rule set demanding no dependencies, no triggers, no reliances, and no diminishing or determinating factors. The original proposal was explicitly non-metaphysical: an experiment to scale a defined law inside a simulated environment and discover its limitations, variations, and breakage points, without asserting anything about physical reality.^{[[^]1]}

2.2 First Response Pass-Through (15 August 2026)

Six AI systems independently produced structured first-pass concept documents in response to the same raw framing:^{[[^]1]}

Contributor	Working title	Primary contribution
Grok (xAI)	Law Escalation Integrity Laboratory (LEIL)	Four-layer architecture (Environment, Law Instantiation, Escalation Engine, Probe & Violation Suite); explicit non-goals list; high-level ten-step test cycle [[^] 1]
Perplexity AI	Ascension and Evolution Constraint Laboratory	Five-layer architecture (Specification, World, Execution, Observation, Challenge); full failure taxonomy; graded independence spectrum; phased plan; documentation template; Experiments A–F [[^] 1]

Contributor	Working title	Primary contribution
Microsoft Copilot	Law Stress-Test Lab (LSTL)	Multidimensional escalation model (intensity, breadth, depth, cardinality, recursion); environment archetypes; reusable law test template [^1]
Google Gemini	The Limitless Protocol	Four-phase adversarial methodology (Initialization, Infinite Escalation, Collision/Paradox, Post-Mortem); "Non-Derived Paradox" framing [^1][2]
Claude AI	Law Scaling & Fracture Lab (LSFL)	Cleanest reusable procedure; result taxonomy (unbounded-clean, unbounded-dependent, asymptotic, hard-capped, fracture-prone, self-contradictory); sharpest open questions [^1]
ChatGPT	(meta-framework, unnamed)	Distinction between "unbounded" and "unspecified"; entity→environment→simulation→laboratory authority hierarchy; hypothesis that many prior "laws" are operators applicable to a smaller Law Transformation Grammar [^1]

What survived independent formalization: the boundary-mapping reframe (map where a law breaks rather than prove it limitless), the four-to-five-layer architecture pattern, treatment of "no dependencies" as aspirational rather than literal, and the instinct to begin small rather than with transfinite constructs. What did not fully survive scrutiny in the second pass: Gemini's literal "infinite escalation" and assumed-crash methodology, Claude's "scales indefinitely" classification language, and Copilot's literal finite-to-uncountable cardinality dial — all three were flagged as overclaiming relative to what finite execution can actually demonstrate.[^2][1]

2.3 Decision and Red Team Pass (16 August 2026)

Five systems (ChatGPT, Claude, Gemini, Copilot, and Grok) independently reviewed the First Response Pass-Through and converged on a decision-and-critique agenda rather than new concept variants. Key developments in this round:[^2]

- ChatGPT introduced the D0–D4 dependency spectrum, the A0–A6 authority-domain hierarchy, a seven-layer architecture (adding Semantic Analysis as its own layer), the "Law Identity Escape" and "Epistemic Failure" failure modes, an Assumption Register with Assumption Inversion Testing, and the L-001 "Unbounded Successor Law" pilot proposal. [^2]
- Claude identified genuine (not merely stylistic) divergences requiring a team decision: how literal the transfinite/uncountable track should be, differing depths of formalism across contributors, and Gemini's "Collision Phase" as a distinct, valuable probe type rather than a default methodology.[^2]
- Gemini reversed part of its own first-pass framing, explicitly recommending discarding Claude's "unbounded-clean" classification language and raising the "Non-Derived Paradox" — the observation that placing an absolutely non-derived, non-contingent law into any simulation substrate already imposes a contingency on it.[^2]
- Copilot produced the most systematic list of unresolved definitional gaps (law, environment, inheritance, ascension, failure taxonomy, escalation model, governance) and explicit "before the next pass" checklist.[^2]
- Grok synthesized all five second-round reviews and issued a "Recommended Immediate Team Actions" list: lock a minimal shared glossary, decide the status of the "no dependencies" doctrine, agree on the first executable prototype, separate the executable

and symbolic tracks explicitly, adopt a lightweight result taxonomy, and preserve all six first-pass responses as versioned source material.^{[1][2]}

2.4 Original proposal versus what survived formalization

Original element	Status after two rounds of independent review
"No dependencies / no triggers / no reliances / no diminishing factors" as an absolute rule	Unanimously reframed as a graded, testable condition (D0–D4 spectrum); absolute version deemed incoherent for any executable simulation ^{[1][2]}
Scale a law "as far as it can go"	Reframed as escalation along named axes (magnitude, breadth, depth, recursion, cardinality) with predefined stop conditions, not literal infinite escalation ^[^2]
Transfinite / transcardinal / uncountable ascension	Split into an executable track (finite approximation) and a symbolic/formal track (theorem-style reasoning); literal execution of uncountable processes rejected as impossible on finite hardware ^{[1][2]}
"This is not a claim about reality"	Preserved unanimously and unchanged across all eleven contributor passes ^{[1][2]}
Consolidate evolution/ascension variants into one experiment	Preserved, but reframed as candidate subjects/operators to be tested individually before cross-law composition ^{[1][2]}
Single unified "law" concept	Fractured into a proposed taxonomy of subtypes (constraint, permission, obligation, transformation, persistence, escalation, inheritance, meta-law, operator) requiring a team decision ^[^2]

3. Mission Statement, Purpose, Objectives, and Core Research Questions

3.1 Mission Statement

The Law Laboratory is a controlled, non-metaphysical simulation research program that formally specifies candidate "laws" governing transformation, escalation, ascension, and inheritance; instantiates them inside declared, minimal-assumption environments; subjects them to systematic escalation and adversarial challenge; and produces auditable, versioned records of the conditions under which each law remains coherent, requires a dependency, reaches a bound, or fails — treating every such finding, including negative and boundary results, as the laboratory's primary scientific output.^{[1][2]}

3.2 Purpose of the Laboratory (precise, non-metaphysical)

The laboratory exists to convert previously informal, rhetorically extreme constructs (infinite ascension, unbounded evolution, absolute independence) into formally specified, falsifiable objects that can be executed or symbolically analyzed under declared assumptions. It makes no claim that any simulated law describes physical reality, biological systems, computation generally, or society, and no simulation result may be interpreted as evidence about base reality. Its outputs are boundary maps — documented statements of what a given formalized law does under specific, declared conditions — not proofs of universal truth.^{[1][2]}

3.3 Primary Objectives (numbered, measurable)

1. **Establish a locked, shared vocabulary** covering law, environment, entity, state, escalation, dependency, trigger, inheritance, ascension, breakage vector, invariant, and authority domain, ratified by all active team members before any executable run.[¹2]
2. **Formally classify every candidate law** into one of the recommended subtypes (constraint, permission, obligation, transformation, persistence, escalation, inheritance, meta-law, operator) using the Candidate Law Record template prior to simulation.[¹2]
3. **Execute at least one minimal, fully instrumented pilot experiment (L-001)** with predefined stop conditions, control runs, and an assumption register before any recursive, cascading, or high-order variant is attempted.[²]
4. **Produce a dependency map (D0–D4) for every tested law**, distinguishing substrate, declared, hidden operational, environmental, and trigger dependencies, with each dependency's criticality assessed via removal testing.[²]
5. **Classify every experimental outcome using separated failure-mechanism and final-disposition taxonomies**, avoiding single-axis classification that conflates "what went wrong" with "what we now conclude".[²]
6. **Maintain a non-overwrite version history** for every law and every experimental result, such that no fracture, failure, or dissenting probe result is ever silently erased or redefined away.[¹2]
7. **Separate all high-order (transfinite, transcardinal, uncountable) work into a distinct symbolic/formal track**, executed only after the baseline executable methodology is validated on finite cases.[¹2]

3.4 Core Research Questions

- Under what specific, declared conditions does a formally specified law of escalation, ascension, or evolution remain internally coherent, and under what conditions does it fail, require an undeclared dependency, or become undefined ?[¹]
- Is the "no dependencies / no triggers / no reliances" doctrine achievable as a graded property (declared independence, external independence, resource resilience, and so on), or does every executable instantiation necessarily carry an irreducible substrate dependency (D0) ?[²]
- What minimum set of properties (memory, goals, structure, lineage, permissions) must persist across a transformation for the result to count as a continuation of the original entity rather than a successor or replacement ?[¹2]
- Which of the previously catalogued "laws" (recursive, cascading, unbounded, transfinite, and so on) are genuinely independent laws, and which are better modeled as operators applied to a smaller set of base rules (the Law Transformation Grammar hypothesis) ?[¹2]
- How should the laboratory distinguish a law-domain failure (the rule contradicts itself) from an environmental failure, an implementation failure, an instrumentation failure, or an epistemic failure (the evaluator itself has been compromised) ?[²]

- What does "unbounded" mean in a given experiment — an unbounded domain, an unbounded trajectory, an unbounded metric, or unbounded rule authority — and which subtype is actually being tested in any given run ?^[^2]

4. Laboratory Architecture Recommendation

4.1 Synthesis rationale

Three architectures were proposed across the two rounds: Grok's four layers (Environment, Law Instantiation, Escalation Engine, Probe & Violation Suite); Perplexity's five layers (Specification, World, Execution, Observation, Challenge); and ChatGPT's seven layers, which add a Semantic Analysis layer before execution and split Observation from Classification. Copilot, Gemini, and Claude did not propose materially different layer counts but used different names for equivalent functions. The seven-layer model subsumes the other two without losing any function identified by any contributor, and it uniquely adds pre-execution static analysis of the law's wording — a capability none of the four- or five-layer models provide and which can reject an ill-specified law before any computational resources are spent. The recommended architecture below adopts the seven-layer model as the baseline, explicitly cross-referencing which earlier contributor's function each layer absorbs.^{[1][2]}

4.2 Recommended seven-layer architecture

Layer	Function	Absorbs / supersedes
1. Specification Layer	Defines the law, its type (constraint, permission, obligation, transformation, persistence, escalation, inheritance, meta-law, operator), scope, invariants, declared dependencies, authority domain, and falsification conditions	Grok's Law Instantiation Layer; Perplexity's Specification Layer; Copilot's Law Definition Layer ^{[1][2]}
2. Semantic Analysis Layer	Attacks the specification's wording before any execution — tests definitional ambiguity, underdetermination, and whether the law can even be operationalized	New; unique to ChatGPT's second-pass proposal ^[^2]
3. Environment Layer	Defines the minimal declared substrate: available state, resources, time model, causality, information access, and competing entities	Grok's Environment Layer; Perplexity's World Layer; Copilot's Environment Design Layer; Gemini's "vacuum state" (renamed and corrected — see Section 5.9) ^{[1][2]}
4. Execution Layer	Instantiates and runs the system reproducibly, with seeded randomness and full event logging	Perplexity's Execution Layer; Copilot's Simulation Layer ^[^1]
5. Escalation Layer	Changes intensity, breadth, depth, recursion, propagation, authority, or state-space assumptions along explicitly named axes with predefined stop conditions	Grok's Escalation Engine; Copilot's escalation dimensions; corrected version of Gemini's Phase II ^{[1][2]}
6. Challenge / Red-Team Layer	Applies contradiction cases, dependency removal, trigger manipulation, inheritance attacks, environment changes, metric attacks, identity attacks, and Gemini's Collision Test (opposing-absolute confrontation) as a distinct probe type	Grok's Probe & Violation Suite; Perplexity's Challenge Layer; Gemini's Collision Phase (reclassified as one probe among several, not the default methodology) ^{[1][2]}

Layer	Function	Absorbs / supersedes
7. Observation & Classification Layer	Maintains independent measurement, full provenance, failure-mechanism classification, final-disposition classification, and limitation-surface mapping	Perplexity's Observation Layer; Grok's archival step; ChatGPT's failure-mechanism/disposition split ^[1] [2]

4.3 Three parallel tracks

The architecture operates across three explicitly separated tracks, a distinction raised independently by Perplexity in the first pass and formalized by ChatGPT and Claude in the second:^[1][2]

Executable Track. Finite computation only: parameter sweeps, recursive models bounded by iteration limits, network propagation, randomized trials, seeded and reproducible. No experiment on this track may report "infinite execution," "scales indefinitely," or "uncountable state processed" — those phrases describe claims the finite hardware cannot support.^[^2]

Symbolic/Formal Track. Ordinal and cardinality arguments, uncountable-structure reasoning, theorem-style proofs, logical models, and formal approximations of transfinite or transcardinal constructs. Results here are proofs or formal arguments, not simulation logs, and must never be merged with Executable Track outputs as if they were the same kind of evidence.^[1][2]

Semantic/Specification Track. Static, pre-execution analysis of a law's wording: what counts as an entity, what does "always" quantify over, does zero available transition space falsify the law, must identity persist, can ascension consist of mere relabeling. A law can fail on this track alone, without ever reaching simulation, and that is a legitimate, resource-saving outcome.^[^2]

4.4 Authority-domain hierarchy

Per ChatGPT's second-pass proposal, every law must declare the authority domain at which it operates, because a rule that appears to have been "violated" may simply have been overridden by a superior experimental layer that was always outside its stated scope:^[^2]

Domain	Scope
A0	Object property
A1	Entity law
A2	Population/system law
A3	Environment law
A4	World/runtime law
A5	Simulation-controller law
A6	Laboratory governance

When a candidate law states "nothing can override this rule," the laboratory must ask: nothing at which layer? A law with entity-scope (A1) sovereignty that is paused or overwritten by the simulation controller (A5) has not necessarily been falsified — the override may simply have occurred at a layer outside the law's declared authority domain.^[^2]

5. Red-Team Analysis

5.1 The "no dependencies / no triggers" doctrine

All eleven contributor passes agree that an executable simulation cannot have literally zero dependencies, because it necessarily depends on a formal representation language, an execution substrate, initial conditions, and computational resources. This is the single most unanimous correction to the original framing and should be treated as settled. The unresolved question is what to do with that correction. ChatGPT's D0–D4 spectrum is the strongest proposed resolution: it does not discard the original doctrine but converts it into a specific, falsifiable test claim — "D1 = 0 and D2 is hypothesized to equal 0 within the law's modeled domain" — which the laboratory then attempts to falsify. This preserves the design pressure of the original rule while removing its incoherent absolute form.^{[1][2]}

Level	Definition
D0	Substrate dependencies — required because a simulator exists at all
D1	Declared law dependencies — explicitly required by the law's own specification
D2	Hidden operational dependencies — discovered only during testing
D3	Environmental dependencies — supplied by the particular world instance
D4	Trigger dependencies — events or thresholds required for activation

Decision required: Is the doctrine a universal constraint applied to every candidate law, a control condition against which laws are compared, a candidate law in its own right, or a null hypothesis to be falsified per law? Copilot's second-pass review lists this as one of ten unresolved questions, and no consensus position emerged.^[^2]

5.2 Definition of "law"

Each of the six first-pass contributors defined "law" differently — a formal statement of permitted transformation (Grok), a rule governing state change (Perplexity), a formal rule or principle constraining behavior (Copilot), an object with a starting condition, scaling curve, and resistance (Claude). Copilot's second-pass review flags this explicitly as the top-priority gap: "You need a single canonical definition before the next pass". ChatGPT's proposed resolution — treating law as a family of subtypes rather than forcing one definition — is the strongest available synthesis, because the alternative (picking one narrow definition) would exclude legitimate candidate subjects such as meta-laws and operators.^{[1][2]}

Subtype	Form
Constraint law	X may never occur
Permission law	X may occur
Obligation law	X must occur when C holds
Transformation law	X becomes Y according to T

Subtype	Form
Persistence law	P survives transformations
Escalation law	Some parameter continues changing under E
Inheritance law	Properties transfer according to I
Meta-law	Governs other laws
Law operator	Modifies a law rather than directly governing entities

The operator category is the highest-leverage idea in either compilation. If recursion, cascading, inheritance, independence, self-modification, and unboundedness are operators rather than separate laws, the project's eight-plus variant families collapse into a small set of composable operators applied to base rules — a "Law Transformation Grammar". Both Claude and Grok's second-pass reviews flag this as the single most interesting idea in the batch, but both explicitly recommend parking it as a dedicated later-stage experiment rather than adopting it immediately, since it has not yet been tested against a working base lab.^{[1][2]}

5.3 Entity identity versus law identity after transformation

Two distinct but structurally parallel identity problems exist. **Entity identity:** if an entity "ascends" and a new state appears with none of the original entity's relevant continuity (memory, goals, structure, lineage, permissions), the claim that the original entity reached the new state may be false — the result may be a successor, not a continuation. **Law identity:** if a law is permitted to recursively rewrite itself, at what point does it stop being the tested law? ChatGPT's second-pass review introduces "Law Identity Escape" as a failure mode: a candidate law that avoids falsification by changing so extensively under stress that the surviving rule can no longer reasonably be classified as the law originally under test. Both problems require the same underlying fix — a declared minimum retention vector (core invariant, lineage, semantic purpose, transformation history, retained clauses, authority scope) that must be checked at every transition.^{[1][2]}

5.4 Authority domain hierarchy

See Section 4.4. The red-team risk here is that, without this hierarchy, the laboratory will systematically over-report "breakage" whenever a superior layer (the simulation controller or the laboratory governance layer) legitimately intervenes in a lower-layer entity or law. ChatGPT's example is exact: a rule stating "nothing can override this rule" is not falsified merely because the simulation pauses or restores the entity, if the rule's declared authority domain was always A1 (entity-scope) rather than A5 (simulation-controller-scope). Locking this hierarchy before the pilot run is a prerequisite for any trustworthy breakage claim.^[^2]

5.5 Simulator failure versus law failure versus instrumentation failure

ChatGPT's second-pass review introduces a six-way failure classification that must be adopted as mandatory, because conflating these categories will produce false breakage reports across the entire program:^[^2]

Failure type	Meaning
Law-domain failure	The rule itself becomes contradictory or impossible under its own definitions
Environmental failure	The world no longer supplies conditions the law requires
Implementation failure	The simulator cannot represent or calculate the required state
Instrumentation failure	The result cannot be measured, even though it occurred
Experimental-design failure	The protocol cannot determine what conclusion follows from the data
Semantic failure	The specification itself becomes undefined at some point

A recursive process exhausting available memory, for example, may indicate the law hit a genuine limit, or may simply indicate the implementation hit a limit — these must never be reported as equivalent findings.[^2]

5.6 Risk of overclaiming "unbounded" or "infinite" results

This is a recurring, unanimous red-team concern across the second pass. ChatGPT proposes a hard rule: no experiment may report "infinite execution" when the actual experiment performed finite computation, and lists acceptable substitute phrasings ("no ceiling specified," "no failure observed through N iterations," "symbolic unbounded model," "asymptotic continuation," and so on). Gemini's second-pass review independently arrives at the same conclusion from a different angle, explicitly recommending that Claude's "unbounded-clean" classification be discarded because it misrepresents what a finite test can establish. ChatGPT further distinguishes four non-interchangeable subtypes of "unbounded" that must each be separately declared per experiment:[^2]

Subtype	Meaning
Unbounded domain	No predefined maximum possible state exists
Unbounded trajectory	The transformation process can always produce another valid state
Unbounded metric	A measured quantity has no upper bound
Unbounded rule authority	The law can continually expand what it governs

An entity can have an unbounded metric while occupying a finite number of meaningful structural states; a law can permit unlimited iterations that merely cycle among three states. Conflating these subtypes will produce misleading headline results.[^2]

5.7 Evaluator independence and epistemic corruption risk

Perplexity's first-pass response already warns against systems that self-grade their own success criteria. ChatGPT's second-pass review elevates this into a dedicated failure category — "Epistemic Failure": the laboratory can no longer reliably determine what occurred, even while the underlying simulation remains perfectly operational, because the subject has altered its success metric, its evaluator, its logs, its history, or what the laboratory can observe. This risk becomes acute once Recursive Law Modification experiments begin (Phase 4 in the roadmap

below), and the laboratory must decide in advance whether an experimental subject may ever modify its own evaluator, and if so, under what containment.[¹][2]

5.8 Governance and adjudication when AI collaborators disagree

This is the most significant unresolved gap identified across the entire second pass. Grok's first-pass framework states that "no authority [is] granted to any single AI or human participant to declare a result final," but no contributor in either pass specifies who adjudicates when independent red-team probes on the same law produce conflicting fracture-search results. Claude's second-pass review raises this explicitly as an open question, alongside a related, previously unaddressed question: if a fracture result is found in a law that is already canon-locked elsewhere in the broader Omniversal Architect Labs body of work, does the fracture override canon, sit alongside it as a documented limitation, or trigger a formal revision cycle? Neither question was resolved by any contributor and must be settled before the pilot's results are treated as final.[¹][2]

5.9 Gemini's "Non-Derived Paradox" and vacuum-state framing

Gemini's second-pass review raises a genuinely novel point: testing a concept defined as absolutely non-derived and non-contingent inside any simulation substrate already imposes a dependency (the substrate itself) on an entity that by definition has none. This is a specific, valuable instance of the general D0 substrate-dependency problem (Section 5.1) rather than a separate issue, and should be filed as the first candidate test case for the Assumption Inversion methodology (Section 6, Assumption Register) rather than treated as requiring an "inverted" architecture. Separately, ChatGPT's second-pass review directly corrects Gemini's first-pass "empty simulation void of any other rules" framing: an executable environment cannot literally contain no rules, since state, transitions, and representation already constitute structure. The recommended terminology is "minimal declared substrate," not "vacuum state" or "void."[^2]

5.10 Additional definitional risks not yet fully addressed

Copilot's second-pass review identifies that "environment," "inheritance," and "ascension" each still lack a single locked definition across contributors, despite repeated use in escalation and boundary claims. These gaps are addressed in the Unified Glossary (Section 6) but remain unresolved decisions requiring explicit team ratification, not analyst fiat.[^2]

6. Unified Glossary (Proposed)

The following definitions synthesize the strongest available treatment from each contributor. Minority positions are noted where a viable alternative remains.

Law. A formally specified rule governing permitted or required behavior of an entity, environment, or other law, classified into one of nine subtypes: constraint, permission, obligation, transformation, persistence, escalation, inheritance, meta-law, or operator. *Minority position:* Grok's narrower first-pass definition ("a formal statement of permitted transformation or persistence") remains viable for laws that are provably confined to those two subtypes and may be used as shorthand in that restricted case.[¹][2]

Environment. The minimal declared substrate hosting entities, states, and transformations, including available resources, time model, state-space size, information access, randomness, competing agents, and evaluation criteria — as free of undeclared assumptions as achievable, with every environmental rule that is not explicitly declared treated as a potential contaminant.[¹][2]

Entity. Anything that can occupy a state and change under a law: an agent, process, species, data structure, rule set, or symbolic object.[¹]

State. The measurable condition of an entity at a given iteration, represented as a vector of declared variables selected per experiment (for example, capability, resources, information, memory, inheritance score, dependency load) rather than a fixed universal set.[¹]

Escalation. A systematic, axis-specific increase in a law's magnitude, breadth, depth, recursion, or cardinality, executed stepwise with predefined stop conditions; never an unqualified move toward "as far as possible" or "infinite".[¹][2]

Dependency (D0–D4 spectrum). A required condition for a law's operation, graded as: D0 substrate dependency (required because a simulator exists), D1 declared law dependency (explicitly stated by the law), D2 hidden operational dependency (discovered during testing), D3 environmental dependency (supplied by the particular world instance), D4 trigger dependency (an event or threshold required for activation).[²]

Trigger. A specific condition that activates a state transition, evaluated for whether it is explicit or hidden, necessary or incidental, stable or fragile, reproducible, vulnerable to manipulation, or a source of circular reasoning.[¹]

Inheritance / Continuity. The set of properties (memory, goals, structure, constraints, rights, permissions, lineage) that must persist across a transformation for the result to count as a continuation of the original entity or law rather than a successor or replacement; requires a declared minimum retention threshold per experiment.[¹][2]

Ascension. A specified movement from one declared class of state to another according to predefined criteria (capability threshold, minimum inheritance retention, dependency ceiling, viability floor, reproducibility), never an unqualified claim of "becoming better".[¹]

Breakage Vector. A precise, reproducible way in which a law is violated, rendered inconsistent, or forced into contradiction while still nominally claiming to be the original law.[¹]

Invariant. A property that must remain true throughout a run and that no in-run process (including recursive self-modification) may alter; the "constitution layer" of a given experiment.[²][1]

Authority Domain. The declared scope (A0 object property through A6 laboratory governance) at which a given law operates, used to determine whether an apparent violation occurred within the law's own scope or via a legitimate override from a superior layer.[²]

Unbounded (four subtypes). Unbounded domain (no predefined maximum state), unbounded trajectory (a valid successor state can always be produced), unbounded metric (a measured quantity has no upper bound), and unbounded rule authority (the law can continually expand

what it governs) — never used interchangeably, and never reported without specifying which subtype and over what tested range.[^2]

7. Proposed Failure & Disposition Taxonomy

Per ChatGPT's second-pass recommendation, failure mechanism (what went wrong) must be reported separately from final disposition (what the laboratory now concludes).[^2]

7.1 Failure mechanisms

Mechanism	Meaning	Example
Direct violation	Law enters an explicitly prohibited state	A "non-dependent" agent requires an undeclared external oracle [^1]
Constraint breach	Law works only by exceeding a stated limit	Infinite growth requires unlimited memory in a finite-budget world [^1]
Hidden dependency (D2)	Behavior relies on an unstated support	Progress stops when a background evaluator is removed [^1]
Circularity	Law assumes the property it claims to generate	"Ascension occurs when an ascended state is reached" [^1]
Undefined transition	No rule specifies an edge case	Two ascension paths conflict with no tie-breaking rule [^1]
Inheritance failure	Claimed continuity is not preserved	Successor retains no memory, values, or identity link [^1]
Metric failure	"Improvement" cannot be measured independently	System grades itself as successful without external criteria [^1]
Runaway escalation	System expands but becomes unstable or nonviable	Growth accelerates while coherence approaches zero [^1]
Degenerate success	Metric technically met in a meaningless way	Score inflates without functional capability gain [^1]
Nontermination	No decidable endpoint reached	Model loops indefinitely without measurable advancement [^1]
Representation failure	Concept cannot be faithfully implemented in the model	An uncountable construct treated as a finite enumerable list [^1]
Law Identity Escape	Law avoids falsification by rewriting itself beyond recognition	Recursive self-modification eliminates the tested clause entirely [^2]
Epistemic Failure	Laboratory can no longer determine what occurred	Subject alters its own evaluator, logs, or observable history [^2]
Implementation failure	Simulator cannot represent or calculate required state	Recursion exhausts available memory before a law limit is reached [^2]
Instrumentation failure	Result occurs but cannot be measured	Observation channel lacks resolution to distinguish two states [^2]
Semantic failure	Specification becomes undefined	"Higher" or "more evolved" lacks an operational definition [^1][2]

7.2 Final dispositions

Disposition	Meaning
Supported within tested bounds	Behaved as predicted under the declared conditions [¹]
Conditionally supported	Worked only under specific declared dependencies or parameter ranges [¹]
No bound observed within tested envelope	No failure occurred through N iterations at declared scale; not a claim of literal unboundedness [²]
Asymptotic within tested range	Approaches a limit without violating it, in the tested envelope [¹]
Hard-capped	A definite ceiling was found and its cause is named [¹]
Fracture-prone	Breaks under specific, named test conditions [¹]
Self-contradictory at scale	Law negates its own premise past a named tier [¹]
Partially specified	Produced behavior, but key terms or edge cases remain undefined [¹]
Insufficiently specified for testing	Rejected at the Semantic Analysis Layer before execution [²]
Rejected in current form	Failed its own predeclared criteria or cannot be made operationally coherent [¹]
Symbolically interesting	Cannot be directly executed but is explorable via formal approximation [¹]

8. Pilot Experiment Specification (L-001)

8.1 Selection rationale

ChatGPT's second-pass proposal for a first executable experiment — the "Unbounded Successor Law" — is adopted as the recommended pilot because it isolates the single most foundational claim underlying nearly every later variant (recursive, cascading, unbounded evolution, and ascension): can continuation remain available without a predefined terminal state ? This directly implements Perplexity's and Grok's shared first-pass recommendation to begin with a small, fully measurable prototype rather than a transfinite or omniversal-scale construct.^{[1][2]}

8.2 Law statement

L-001 — Unbounded Successor Law, v0.1: For every valid state S_n , the law permits generation of at least one successor state S_{n+1} that satisfies invariant I . The law is deliberately not named "evolution" or "ascension" at this stage, to avoid importing unexamined connotations before the base mechanism is tested.^[^2]

8.3 Environment

A minimal declared substrate (Section 4.3, Executable Track) with a discrete time step, a finite initial state space, an explicit representation of resources, and full event logging. Per Grok's second-pass recommendation, the pilot should run under a two-stage design: a Common Benchmark Environment (identical initial conditions, for comparability) followed by Specialized Challenge Environments tailored to probe L-001's specific assumptions.^[^2]

8.4 Variables

Variable	Role
Successor availability	Whether at least one valid S_{n+1} exists at each step
Invariant I	Declared property that must hold for any successor to be valid
Inheritance score	Degree of continuity between S_n and S_{n+1}
Resource budget	Finite or resource-limited world setting
Trigger presence	Whether a trigger condition gates the transition
Dependency exposure	D0–D4 classification of any support the law leans on

8.5 Escalation and challenge conditions

Per ChatGPT's proposed test battery for L-001:[^2]

- Finite state space vs. expandable state space
- Resource-limited world vs. unrestricted world
- Inheritance required vs. inheritance removed
- Trigger required vs. trigger removed
- Duplicate successors vs. branching successors
- Recursive successor generation
- Evaluator manipulation attempt
- Environment migration mid-run
- Conflicting competing law introduced
- No valid successor available (boundary case)
- Law attempts to redefine "valid" (Law Identity Escape probe)

8.6 Stop conditions

A run terminates when: no valid successor exists and none can be generated within the declared resource budget; a predefined maximum iteration count is reached with no failure observed (reported as "no bound observed within tested envelope," not "unbounded"); a direct violation, circularity, or representation failure is detected; or the evaluator is found to have been altered by the subject (Epistemic Failure), which halts the run for review rather than continuing to log results.[^2]

8.7 Success and failure criteria (predeclared)

Success (informative outcome, not validation): The law's dependency profile (D0–D4), inheritance requirements, and failure mechanism (if any) are fully characterized and reproducible across seeded runs.

Failure (informative outcome, not project failure): Any of the sixteen failure mechanisms in Section 7.1 is observed and documented with its minimal triggering condition.

8.8 Control runs (mandatory)

Per Perplexity's first-pass recommendation, made mandatory in the second-pass review, L-001 must be compared against: an ordinary fixed-progression control; a random-transformation control; a bounded-recursion control; and the same system with inheritance disabled. Without these controls, any observed successor-generation behavior cannot be attributed to the law rather than to ordinary environmental dynamics.^{[1][2]}

8.9 Assumption register (initial)

ID	Assumption
A-001	Discrete time is assumed
A-002	Identity remains meaningful after transformation
A-003	State transitions are observable
A-004	Evaluator is external to the subject
A-005	Environment remains stable during baseline testing

Each assumption is subject to Assumption Inversion Testing in later phases (for example: what if A-004 is false and the entity gains access to the evaluator).^[^2]

9. Phased Development Roadmap

Phase	Focus	Key deliverables	Primary sources
Phase 0	Charter and glossary lock	Ratified glossary, law-type taxonomy decision, authority-domain hierarchy adoption, dependency-doctrine decision	^{[1][2]}
Phase 1	Minimal executable prototype	L-001 run under Common Benchmark Environment; baseline metrics; first failure report; control-run comparison	^{[1][2]}
Phase 2	Variation matrix	L-001 under Specialized Challenge Environments; resource, inheritance, and trigger sweeps; stability/brittleness map	^{[1][2]}
Phase 3	Adversarial and identity testing	Full Challenge Layer probe suite; Law Identity Escape and Epistemic Failure checks; Assumption Inversion Testing	^[^2]
Phase 4	Recursive and cascading systems	Controlled self-modification with an enforced invariant/constitution layer; multi-entity cascade threshold mapping; evaluator-independence stress test	^{[1][2]}
Phase 5	Operator-grammar exploration	Test whether recursion, inheritance, cascading, and unboundedness function as operators on L-001-class base rules (Law Transformation Grammar hypothesis)	^{[1][2]}
Phase 6	Symbolic high-order track	Formal/theorem-style treatment of transfinite, transcardinal, and uncountable constructs, explicitly separated from executable results	^{[1][2]}

10. Governance Recommendations

Versioning of laws. Every law revision produces a new version number, not a replacement. A fracture discovered in Law U-1 v1.0 under condition C is recorded permanently as "U-1 v1.0 — fracture F-003 under condition C"; a subsequent revision becomes U-1 v1.1, and both versions remain independently queryable.^{[1][2]}

Adjudication of conflicting probe results. No contributor in either pass resolved this; it is flagged here as the single largest open governance gap. Recommended default, pending team ratification: when two collaborators' fracture-search results on the same law disagree, both results are logged as independent probe records under the Observation & Classification Layer, and the disagreement itself becomes an Assumption Inversion Test target rather than being resolved by majority vote or single-contributor authority — consistent with Grok's first-pass principle that no single AI or human participant holds unilateral authority to declare a result final.^{[1][2]}

Relationship of laboratory findings to existing OAL canon. Also unresolved by any contributor and requiring explicit team decision: whether a fracture result on an already canon-locked law (for example, something under an existing Protocol) automatically overrides canon, is logged as a documented limitation alongside canon, or triggers a formal revision cycle. Recommended default, pending ratification: log as a documented limitation alongside canon; never auto-override, since the laboratory's mandate is boundary-mapping, not canon adjudication.^[^2]

Non-overwrite rule for failure history. Failure records, dissenting probe results, and superseded law versions are never deleted or silently redefined. This is unanimous across both passes and should be treated as a hard laboratory rule rather than a recommendation.^{[1][2]}

Provenance and canonical artifact identity. Grok's second-pass review notes a formatting/page-count discrepancy between the PDF and DOCX versions of the first-pass compilation and recommends that future compilations carry a single canonical version identifier and content-integrity indicator so layout differences never create uncertainty about whether two files contain identical intellectual content. This should be adopted as standard practice for all future compiled artifacts.^[^2]

Historical material versus active decisions. Archived first-pass and second-pass AI responses should be clearly distinguished from current, active-team decisions, so a historical contributor response is never later mistaken for a current vote or current authority.^[^2]

11. Recommendations & Decision Items for the Team

11.1 Decisions that must be locked before any code is written

1. Canonical law taxonomy: adopt the nine-subtype model (constraint, permission, obligation, transformation, persistence, escalation, inheritance, meta-law, operator) or specify an alternative.^[^2]
2. Status of "no dependencies / no triggers": universal constraint, test condition, candidate law, or null hypothesis — per the D0–D4 spectrum.^{[1][2]}

3. Minimum inheritance/continuity retention threshold required for a transformed entity to count as a continuation rather than a successor.^{[1][2]}
4. Adoption of the seven-layer architecture and the three-track (Executable / Symbolic / Semantic) separation.^[^2]
5. Adoption of the A0–A6 authority-domain hierarchy.^[^2]
6. Ratification (or amendment) of the L-001 pilot specification in Section 8.^[^2]
7. Adjudication process for disagreeing probe results (no default currently exists; see Section 10).^[^2]
8. Relationship between laboratory findings and existing canon-locked material (no default currently exists; see Section 10).^[^2]
9. Whether the Law Transformation Grammar (operator) hypothesis is tested now as an alternative framing or parked until after the L-001 baseline is validated — current majority recommendation is to park it.^{[1][2]}
10. Single working title for the overall program versus retaining multiple names for different scopes (for example, a general framework name alongside sub-project names like the original LEIL, LSTL, or LSFL).^[^2]

11.2 Dissent and alternative framings to preserve, not force into consensus

- **Gemini's Collision Phase** (deliberately colliding two absolute/infinite laws to force a paradox) remains a valuable probe type and should be folded into the Challenge Layer as one distinct test among several, but should not become the laboratory's default escalate-and-observe methodology, which the majority of contributors favor.^{[1][2]}
- **Gemini's "Non-Derived Paradox"** and inverted-test proposal (subjecting the environment to the law rather than the law to the environment) remains an unresolved alternative architecture that has not been adopted or rejected by the team and should be preserved as a named minority position pending a dedicated review.^[^2]
- **Depth-of-formalism divergence:** Perplexity and Copilot produced ready-to-run specifications, while Gemini and Grok remained conceptual/narrative. This altitude mismatch should be explicitly acknowledged rather than smoothed over in future merges, per Claude's second-pass caution.^[^2]
- **The Law Transformation Grammar (operator) hypothesis** is flagged by both Claude and Grok as the most interesting idea in either compilation, yet it is explicitly not yet adopted; it must remain visible as a live, high-priority future track rather than being absorbed silently into the current taxonomy.^{[1][2]}
- **All six first-pass working titles and architectures** (LEIL, the Ascension and Evolution Constraint Laboratory, LSTL, The Limitless Protocol, LSFL, and ChatGPT's unnamed meta-framework) should remain preserved as versioned source material rather than collapsed into a single "winning" document, per Grok's explicit second-pass recommendation.^[^2]

12. Appendix: Contribution Cross-Reference

Idea	Originating contributor(s)	Round
Four-layer architecture (Environment, Law Instantiation, Escalation Engine, Probe Suite)	Grok	First pass [^1]
Five-layer architecture (Specification, World, Execution, Observation, Challenge)	Perplexity	First pass [^1]
Graded independence spectrum (declared → external → resource → trigger → evaluative → structural → recursive)	Perplexity	First pass [^1]
Full failure taxonomy (direct violation through representation failure)	Perplexity	First pass [^1]
Multidimensional escalation (intensity, breadth, depth, cardinality, recursion)	Microsoft Copilot	First pass [^1]
Result taxonomy (unbounded-clean, asymptotic, hard-capped, fracture-prone, self-contradictory)	Claude	First pass [^1]
Adversarial "shatter" posture and Collision/Paradox phase	Google Gemini	First pass [^1]
"Unbounded" vs. "unspecified" distinction; operator/grammar hypothesis; authority hierarchy (early form)	ChatGPT	First pass [^1]
D0–D4 dependency spectrum	ChatGPT	Second pass [^2]
A0–A6 authority-domain hierarchy (formalized)	ChatGPT	Second pass [^2]
Seven-layer architecture with Semantic Analysis Layer	ChatGPT	Second pass [^2]
Law Identity Escape and Epistemic Failure failure modes	ChatGPT	Second pass [^2]
Assumption Register and Assumption Inversion Testing	ChatGPT	Second pass [^2]
L-001 Unbounded Successor Law pilot proposal	ChatGPT	Second pass [^2]
Failure-mechanism vs. final-disposition separation	ChatGPT	Second pass [^2]
Executable vs. symbolic track question flagged as unresolved decision	Claude	Second pass [^2]
Adjudication and canon-relationship governance gaps identified	Claude	Second pass [^2]
"Non-Derived Paradox" and vacuum-state critique	Google Gemini	Second pass [^2]
Systematic definitional-gap checklist (law, environment, inheritance, ascension)	Microsoft Copilot	Second pass [^2]
Synthesis of all second-pass reviews and "Immediate Team Actions" list	Grok	Second pass [^2]

References

1. [Can you help me with a simulation laboratory design and development? I need you to analyze and review the information from the contributors and provide a consolidated action plan. I need you do detailed research and red team everyone's contributions....

...nts and refinement and address key simulation lab elements that have not been introduced. If I provide you with the files that we're currently working on, I just need an alternative analysis done essentially. Is that something that you can help with?](<https://www.perplexity.ai/search/68b042cc-9dce-44e8-8e06-d795ec69379a>) - Yes — I can absolutely help with that. I can review the contributors' materials, stress-test assumpt...

2. [Dear perplexity ai, I have some information I would like to share with you that I need turned into a comprehensive document. the first is titled Kinetic Threshold Demonstration & Open Ocean Parley - Canon Scene Design Dossier. the second will be Netw...

...ernal conflict.

2. Tech vs Emotion Imbalance:

- Risk: heavy emphasis on technology over emotional stakes.
- Mitigation: in both scenes, anchor at least one personal relationship beat and one value conflict (safety vs control, loyalty vs truth, etc.).](<https://www.perplexity.ai/search/d4989fda-8c74-46a7-b396-0dc54c9e3fb4>) - OAL-DOSSIER-CANON-001 v1.0.0 is ready in all three formats — PDF, DOCX, and Markdown.

Here's what t...